

# Cahier des charges du projet

## Interface d'analyse et d'aide à la décision pour les jeux de fantasy cyclisme

Groupe — ZHANG Yuxiang , Alsafadi Kenan, De Michiel Gildas

18 février 2026

### Table des matières

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                 | <b>2</b> |
| <b>2</b> | <b>Objectifs du projet</b>                          | <b>2</b> |
| <b>3</b> | <b>Besoins fonctionnels</b>                         | <b>2</b> |
| 3.1      | Import et gestion des données . . . . .             | 2        |
| 3.2      | Modélisation du problème d'optimisation . . . . .   | 3        |
| 3.3      | Analyse des résultats . . . . .                     | 3        |
| 3.4      | Interface utilisateur . . . . .                     | 3        |
| <b>4</b> | <b>Contraintes techniques et non fonctionnelles</b> | <b>3</b> |
| 4.1      | Environnement de développement . . . . .            | 3        |
| 4.2      | Performances . . . . .                              | 4        |
| 4.3      | Documentation . . . . .                             | 4        |
| 4.4      | Qualité . . . . .                                   | 4        |
| <b>5</b> | <b>Solution envisagée</b>                           | <b>4</b> |
| 5.1      | Modélisation mathématique . . . . .                 | 4        |
| 5.2      | Stratégie de résolution . . . . .                   | 5        |
| 5.3      | Architecture logicielle . . . . .                   | 5        |
| <b>6</b> | <b>Organisation et planification</b>                | <b>5</b> |
| 6.1      | Calendrier prévisionnel . . . . .                   | 5        |
| 6.2      | Livrables . . . . .                                 | 7        |
| <b>7</b> | <b>Conclusion</b>                                   | <b>7</b> |

# 1 Introduction

Les jeux de fantasy cyclisme consistent à composer une équipe de coureurs sous contrainte budgétaire afin de maximiser les points gagnés tout au long d’une course, comme le Tour de France. Le jeu “*On connaît nos classiques, le jeu*” (OCNC), organisé par la RTBF, sert de référence pour ce projet. Les joueurs doivent sélectionner 14 coureurs avec un budget de 140 millions d’euros, en respectant une limite de trois coureurs par équipe réelle. Des règles spécifiques encadrent les transferts (limités aux journées de repos, avec un surcoût de 10 %) et l’utilisation d’un joker unique permettant de doubler les points d’un coureur sur une étape.

Ce projet, encadré par Thibaut Lust, vise dans un premier temps à développer une interface permettant une analyse *post-optimale* : étant donné les résultats réels des étapes (points marqués par chaque coureur), on souhaite déterminer, pour chaque jour de course, quelle composition d’équipe aurait été optimale sous les contraintes budgétaires et de transferts limités. Cette analyse permettra d’étudier la stabilité de l’équipe optimale, les dépendances entre coureurs, et d’identifier des stratégies pour les éditions futures.

## 2 Objectifs du projet

Les objectifs principaux sont les suivants :

- **Analyse post-optimale** : À partir des données réelles d’une course (points par étape, maillots, abandons, etc.), calculer l’équipe optimale *ex post* pour chaque étape, en prenant en compte les possibilités de transferts et l’utilisation du joker.
- **Étude de la stabilité** : Déterminer quels coureurs sont systématiquement présents dans l’équipe optimale, et analyser l’évolution de la composition au fil des étapes.
- **Analyse de dépendances** : Mettre en évidence d’éventuelles corrélations ou complémentarités entre coureurs (par exemple, un coût élevé pour un coureur peut conditionner la présence d’un autre moins cher).
- **Aide à la décision pour les jeux futurs** : Sur la base des analyses précédentes, proposer des stratégies de sélection avant le début d’une course, en tenant compte des incertitudes sur les performances.

Le projet se déroulera en deux phases : une première phase de modélisation et d’implémentation d’un solveur exact (programmation linéaire en nombres entiers) pour le problème avec transferts, et une seconde phase d’exploration d’approches heuristiques si la taille du problème rend la résolution exacte trop coûteuse.

## 3 Besoins fonctionnels

Les fonctionnalités attendues sont les suivantes :

### 3.1 Import et gestion des données

- Lecture automatique des fichiers CSV contenant les prix des coureurs (`valeurs.csv`), les résultats des étapes (`stage_XX_2025.csv`), les attributions de maillots par étape (`maillots.csv`) et les bonus finaux (`final.csv`).
- Possibilité d’intégrer d’autres jeux de données (autres courses, autres règles).

- Gestion des abandons et disqualifications (impact sur les transferts).

## 3.2 Modélisation du problème d’optimisation

- Implémentation d’un modèle de programmation linéaire en nombres entiers (PLNE) représentant le problème de sélection d’équipe sur plusieurs étapes avec transferts limités.
- Prise en compte des règles spécifiques d’OCNC :
  - Budget initial fixe (140 M€).
  - Limite de 3 coureurs par équipe réelle.
  - Transferts possibles uniquement lors des deux fenêtres de repos (après les étapes 9 et 15), avec un surcoût de 10 % sur le prix d’achat des nouveaux coureurs.
  - Joker unique permettant de doubler les points d’un coureur sur une étape choisie.
- Le modèle doit pouvoir être exécuté pour un nombre variable d’étapes et un nombre de coureurs.

## 3.3 Analyse des résultats

- Affichage de la composition optimale pour chaque étape (ou pour chaque fenêtre sans transfert).
- Calcul et visualisation de la stabilité : fréquence de sélection de chaque coureur, taux de renouvellement de l’équipe.
- Détection de corrélations : par exemple, analyse des paires de coureurs souvent sélectionnés ensemble.
- Export des résultats (tableaux, graphiques) pour les inclure dans un rapport.

## 3.4 Interface utilisateur

Bien que l’accent soit mis sur l’analyse, une interface en ligne de commande (ou éventuellement une interface graphique simple) permettra de lancer les calculs et de visualiser les résultats. L’interface devra être documentée et facile à utiliser.

# 4 Contraintes techniques et non fonctionnelles

## 4.1 Environnement de développement

- Langage : Python 3.x.
- Solveur d’optimisation : Gurobi (licence académique disponible).
- Gestion de versions : Git, avec dépôt hébergé sur [https://github.com/Kapiloten/PAI2D\\_Fantasy\\_Cyclisme](https://github.com/Kapiloten/PAI2D_Fantasy_Cyclisme). Tous les documents (code, documentation, cahier des charges) y seront déposés.
- Bibliothèques supplémentaires : pandas pour la manipulation de données, éventuellement matplotlib pour les graphiques.

## 4.2 Performances

- Le temps de résolution pour l'ensemble des 22 étapes avec les données complètes ne devra pas dépasser quelques minutes sur une machine standard.
- Des tests de montée en charge seront effectués : nous évaluerons le temps de calcul en fonction du nombre de coureurs et du nombre d'étapes, afin de déterminer la limite à partir de laquelle une approche heuristique devient nécessaire. Une première expérimentation avec Gurobi sur des sous-ensembles de données sera menée.

## 4.3 Documentation

- Le code sera commenté et accompagné d'un fichier README expliquant l'installation et l'utilisation.
- Une documentation technique décrivant le modèle mathématique sera fournie.
- Les résultats des analyses seront présentés dans un rapport final.

## 4.4 Qualité

- Le code devra être modulaire pour permettre l'ajout de nouvelles contraintes ou variantes.
- Les tests unitaires vérifieront la cohérence des résultats sur des cas simples.
- Respect des délais : livraison du code final avant la date limite de l'UE.

# 5 Solution envisagée

## 5.1 Modélisation mathématique

Le problème est un problème de sac à dos multi-étapes avec transferts, similaire à celui étudié dans les références fournies [1, 2, 3]. Nous proposons un modèle PLNE basé sur les variables binaires suivantes :

- $x_{i,t}$  : 1 si le coureur  $i$  est dans l'équipe au début de l'étape  $t$  (avant d'éventuels transferts à cette étape).
- $y_{i,t}$  : 1 si le coureur  $i$  est transféré (acheté) lors de la fenêtre précédant l'étape  $t$ .
- $z_{i,t}$  : 1 si le coureur  $i$  est transféré (vendu) lors de cette même fenêtre.
- $joker_i$  : 1 si le joker est utilisé sur le coureur  $i$  (un seul coureur sur toute la course).

La fonction objectif maximise la somme des points marqués par les coureurs présents à chaque étape, en doublant les points de l'étape où le joker est activé. Les contraintes assurent :

- La cohérence des effectifs (exactement 14 coureurs à chaque étape).
- Le respect du budget : le coût total des achats initiaux et des transferts (avec majoration de 10 % pour les entrants) ne dépasse pas le budget initial, et le budget évolue avec les ventes (revente au prix d'achat initial).
- La limitation des transferts aux fenêtres autorisées (journées de repos).
- La règle des 3 coureurs maximum par équipe réelle.
- L'utilisation unique du joker.

Ce modèle sera implémenté avec Gurobi. Une attention particulière sera portée à la formulation pour limiter le nombre de variables et de contraintes, afin de garantir des temps de résolution acceptables.

## 5.2 Stratégie de résolution

Dans un premier temps, nous utiliserons Gurobi en tant que solveur exact. Si la résolution du problème complet s'avère trop lente (au-delà de quelques minutes), nous envisagerons :

- Une réduction de la taille du problème par élagage des coureurs dominés (coût élevé pour peu de points).
- L'utilisation de techniques de décomposition (par exemple, relaxation lagrangienne).
- Le développement d'une heuristique (gloutonne, recherche locale) pour fournir une solution approchée en temps raisonnable.

Des tests de performance seront menés pour déterminer le seuil à partir duquel une heuristique devient nécessaire.

## 5.3 Architecture logicielle

Le code sera organisé en modules :

- `data_loader.py` : chargement et prétraitement des données.
- `model.py` : construction du modèle PLNE.
- `solver.py` : lancement de la résolution et récupération des résultats.
- `analysis.py` : fonctions d'analyse et de visualisation.
- `main.py` : interface en ligne de commande.

Cette structure permettra de tester indépendamment chaque composant et de remplacer facilement le solveur exact par une heuristique.

# 6 Organisation et planification

Le projet se déroule sur le premier semestre de l'UE, avec des réunions bimensuelles avec l'encadrant.

## 6.1 Calendrier prévisionnel

- **23 janvier 2026** : Première réunion, présentation des règles et des objectifs. D'ici la prochaine réunion, réalisation de la modélisation sans transfert et obtention de la solution optimale.
- **12 février 2026** : Deuxième réunion, présentation du modèle sans transfert et de sa solution optimale, puis discussion sur la modélisation avec transferts. Pour la prochaine réunion, nous devons tester l'efficacité de Gurobi sur ce problème de fantasy cyclisme en générant aléatoirement des données (prix et scores pour un nombre variable de coureurs, ainsi que plusieurs périodes). Cela permettra d'évaluer à partir de quelle taille de problème le solveur devient trop lent, et de décider si une approche heuristique est nécessaire pour la suite.

- **24 février 2026** : Troisième réunion, présentation des résultats des tests de performance de Gurobi (seuil de passage à l’heuristique) et exposition du modèle avec transferts que nous avons défini (en tenant compte des abandons), sans le joker dans un premier temps. Tests sur les données de 2025.
- **Mars 2026** : Amélioration et finalisation du code, intégration du joker. Nous poursuivrons le développement du modèle en y intégrant la gestion du joker. Cette phase comprendra :
  - La modélisation mathématique du joker dans le cadre PLNE (variable binaire supplémentaire, doublement des points sur l’étape choisie, contrainte d’unicité).
  - L’implémentation dans le code Python avec Gurobi, en veillant à ne pas dégrader les performances.
  - Des tests sur les données 2025 pour valider le bon fonctionnement du joker et comparer les compositions optimales avec et sans joker.
  - L’optimisation du code (par exemple, réduction du nombre de variables, pré-calculs) pour améliorer les temps de résolution.

À l’issue de ce mois, nous disposerons d’un modèle complet incluant les transferts et le joker, prêt pour les analyses.

- **Avril 2026** : Analyses post-optimales (stabilité, dépendances). Début de la réflexion sur l’aide à la décision pour les jeux futurs. Nous exploiterons le modèle finalisé pour réaliser une série d’analyses approfondies sur les données réelles (par exemple, Tour de France 2025) :
  - **Stabilité de l’équipe optimale** : Calcul de la composition optimale pour chaque étape (ou fenêtre sans transfert) ; identification des coureurs « incontournables » (présents dans un grand nombre d’optima) ; mesure du taux de renouvellement de l’équipe entre étapes.
  - **Dépendances entre coureurs** : Analyse des co-occurrences (paires de coureurs souvent sélectionnés ensemble) ; recherche de corrélations négatives (coureurs qui s’excluent mutuellement pour des raisons budgétaires ou de règles d’équipe) ; visualisation sous forme de graphes ou de matrices de corrélation.
  - **Impact du joker** : Étude de l’influence du joker sur la composition optimale et sur le score total ; identification des étapes et des coureurs où son utilisation est la plus bénéfique.
  - Parallèlement, nous amorcerons une réflexion sur l’extension de ces analyses à des contextes prédictifs : comment utiliser les résultats post-optimaux pour éclairer les choix avant une course (par exemple, détection de profils de coureurs sous-évalués, stratégies de gestion de budget, anticipation des besoins de transferts) ?

Les résultats seront synthétisés sous forme de graphiques et de tableaux pour faciliter leur interprétation.

- **Mi-avril 2026** : Quatrième réunion, validation des analyses. Nous présenterons à l’encadrant les résultats des analyses post-optimales (stabilité, dépendances, impact du joker). Cette réunion permettra de :
  - Valider la pertinence des observations et des méthodes employées.
  - Discuter des premières pistes pour l’aide à la décision en contexte futur.
  - Recueillir des retours pour affiner les analyses ou orienter les travaux restants (notamment pour le rapport final).

En fonction des commentaires, nous ajusterons nos analyses ou en développerons de nouvelles.

- **Mai 2026** : Finalisation du code, rédaction du rapport final, préparation de la soutenance. Ce dernier mois sera consacré à :
  - La finalisation et le nettoyage du code source : ajout de commentaires, documentation, tests unitaires, README.
  - La rédaction du rapport final, qui comprendra :
    - Une introduction présentant le contexte et les objectifs.
    - Une description détaillée de la modélisation mathématique (variables, contraintes, fonction objectif).
    - L’architecture logicielle et les choix d’implémentation.
    - Les résultats des tests de performance et des analyses post-optimales (avec graphiques et tableaux).
    - Une discussion sur les limites et les perspectives (notamment l’extension à l’aide à la décision prédictive).
    - Une conclusion résumant les apports du projet.
  - La préparation de la soutenance orale : synthèse des points clés, supports visuels, répétition.
- **Fin mai 2026** : Date limite de remise du projet. Nous déposerons la version finale du code et du rapport sur le dépôt Git avant la date butoir. Le lien sera communiqué aux encadrants par mail ou via Mattermost.

## 6.2 Livrables

- Code source complet (Python) avec documentation.
- Cahier des charges (ce document).
- Rapport final détaillant la modélisation, les résultats des analyses et les recommandations stratégiques.
- Dépôt Git contenant tous les fichiers, accessible à l’encadrant.

## 7 Conclusion

Ce cahier des charges fixe le cadre du projet d’interface d’analyse pour les jeux de fantasy cyclisme. Il a été établi en étroite collaboration avec l’encadrant et tient compte des premières expérimentations réalisées. La solution proposée allie rigueur mathématique (modélisation PLNE) et souplesse (adaptation à la taille du problème par heuristiques si nécessaire). Les prochaines étapes consisteront à implémenter le modèle complet et à réaliser les premières analyses post-optimales sur les données du Tour de France 2025.

## Références

- [1] J. Belien, D. Goossens, D. Van Reeth, “Optimization modelling for analyzing fantasy sport games”, *INFOR : Information Systems and Operational Research*, 2017.

- [2] M. Ausloos, “Should one (be allowed to) replace the Cipollini’s?”, *Annals of Operations Research*, 2024.
- [3] E. Bampis, B. Escoffier, A. Teiller, “Multistage knapsack”, *Journal of Computer and System Sciences*, 2022.